

Feladat leírása

Klasszikus keresés egy tömbben. Egy n elemű tömb legkisebb elemének megkeresése k párhuzamos szál használatával.

Algoritmusok

1. Soros (Serial)

- **Algoritmus név:** `SerialMin`
- Idő komplexitás: $O(n)$
- Hely komplexitás: $O(1)$
- **Leírás:** A klasszikus, egyetlen szálon futó lineáris keresés. Egy `min_val` változóban tartja az eddig talált legkisebb értéket, és végigiterál a tömb összes elemén.
- **Tervezői döntések: * Bázis (Baseline) a mérésekhez:** Ez az algoritmus szolgál viszonyítási alapként. Ennek az idejéhez mérjük a párhuzamos algoritmusok gyorsulását (Speedup).
 - **Fallback stratégia:** A komplexebb párhuzamos algoritmusok (pl. rekurzív tömörítés) a fa legalsó szintjein (amikor a méret egy küszöbérték, pl. 5000-10000 alá csökken) visszaváltanak erre a soros végrehajtásra, mivel ott már a szálkezelés adminisztrációs költsége (overhead) meghaladná a párhuzamosítás hasznát.
 - Konstans referenciaként veszi át a tömböt

2. Szeletelés (Reduction / Chunking)

- **Leírás:** A tömböt k részre (chunk) vágjuk. Minden szál függetlenül, sorosan megkeresi a saját darabjának a minimumát. A legvégén a kapott k darab részeredményt egy végső minimumkereséssel egyesítjük.
- **OMP Reduction (`OmpReduction`)**
 - **Leírás:** Az OpenMP beépített `#pragma omp parallel for reduction(min:min_val)` direktíváját használja, amely a háttérben elvégzi a szeletelést és az összevonást.
 - **Tervezői döntések:** Mivel a ciklusmag csak egy egyszerű összehasonlítás, a **statikus ütemezés** (alapértelmezett) a legcélravezetőbb. A szálak előre megkapják a saját fix szeletüket, így a futás közbeni szinkronizáció (contention) minimális.
- **OMP SIMD (`OmpSimd`)**
 - **Leírás:** A CPU vektoros utasításkészletét (pl. AVX, SSE) használja. A `#pragma omp simd` hatására a processzor regisztereibe egyszerre több elem töltődik be, és egyetlen órajelciklus alatt több összehasonlítás (Single Instruction, Multiple Data) történik.
 - **Tervezői döntések:** Kifejezetten hardverközeli optimalizáció. Kihasználja a modern processzorok architektúráját a memóiafüggetlen (data-parallel) műveleteknél. Kombinálható lenne a szál-szintű párhuzamosítással is.
- **STD Parallel C++17 (`StdParallel`)**

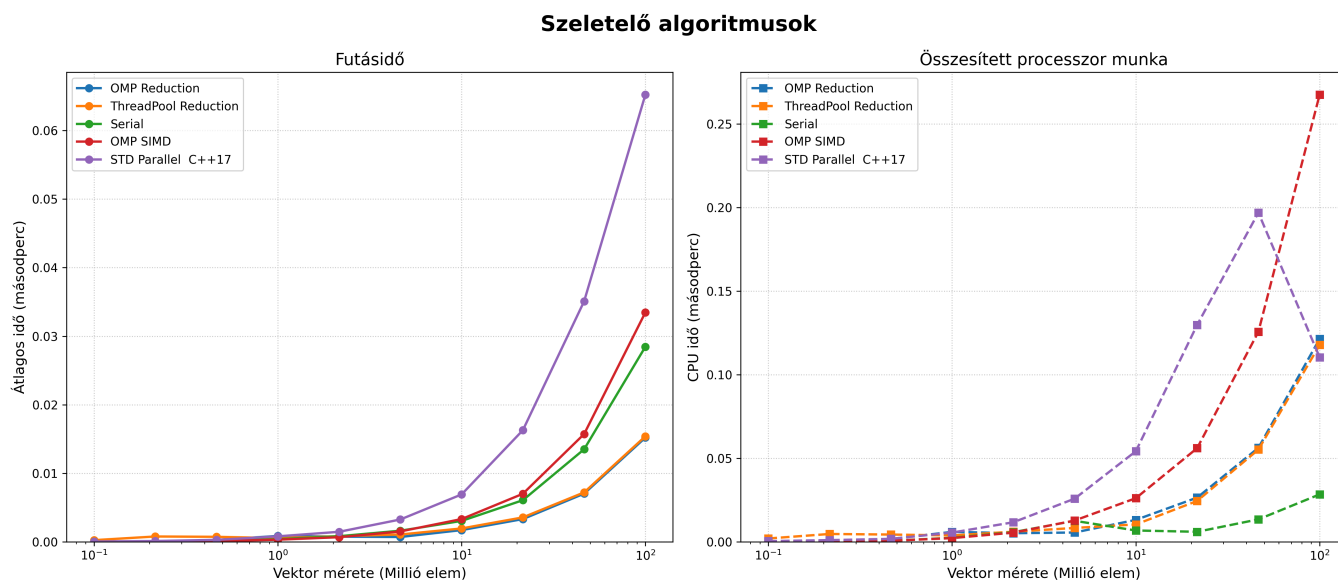
- *Leírás:* A C++17 szabvány beépített párhuzamos algoritmusa:

`std::min_element(execution::par, ...)`. Referenciaként lehet használni a többi implementációhoz.

- **ThreadPool Reduction (`ThreadPoolReduction`)**

- *Leírás:* A szeletelési elv manuális implementációja egyedi szálkészlet (Thread Pool) segítségével. A vektor `num_workers` részre van vágva, a részeredmények egy `partial_mins` tömbbe kerülnek.
- *Tervezői döntések:* A főszál és a worker szálak közötti szinkronizációhoz aktív várakozás (spin-lock) helyett `std::condition_variable` és `std::mutex` használata. Ez garantálja, hogy a főszál nem emészt fel a CPU időt, amíg a többi szál dolgozik (a CPU-Work mérés pontossága miatt kritikus).

Mérések és következtetések:



- A threadpool és az openMP hasonlóan működnek, mert az openMP is a háttérben egy poolt használ, ezért az azonos alapon működő algoritmusok is nagyon hasonló futásidővel rendelkeznek
- Az std parallel min valamennyivel rosszabbul működött a mérési körülmények között, fordítótól és verziótól függően akár más is lehet az algoritmus
- A vektorizációt valószínűleg vagy nem sikerült elindítania az openMP-nek vagy a vektorok mérete lett túl kicsi ahhoz hogy optimális legyen a gyorsítás, bár ez nem valószínű 100 millós vektor hosszánál, mivel az csak 8 részre lett osztva, további lehetőség, hogy valamiért nem sikerült a vektorizáció és a párhuzamosság összehangolása
- Bár az összesített processzor munka több a szeletelő algoritmusoknál így is hatékonyak és az ábrát megközelítőleg lineárisan arányos a soros algoritmusével
 - Bár érdekes, hogy ez a sok többlet cpu idő mihez kellhet, mivel maga az algoritmus elméletben alig igényelne több cpu időt még ha sorban egymás után futnának is a szálak - feltehetőleg a futási környezet, paraméterek és

információk betöltése veszi el a legtöbb időt és ezekhez képes a keresés ideje elhanyagolható

3. Tömörítés (Compression / Tree Reduction)

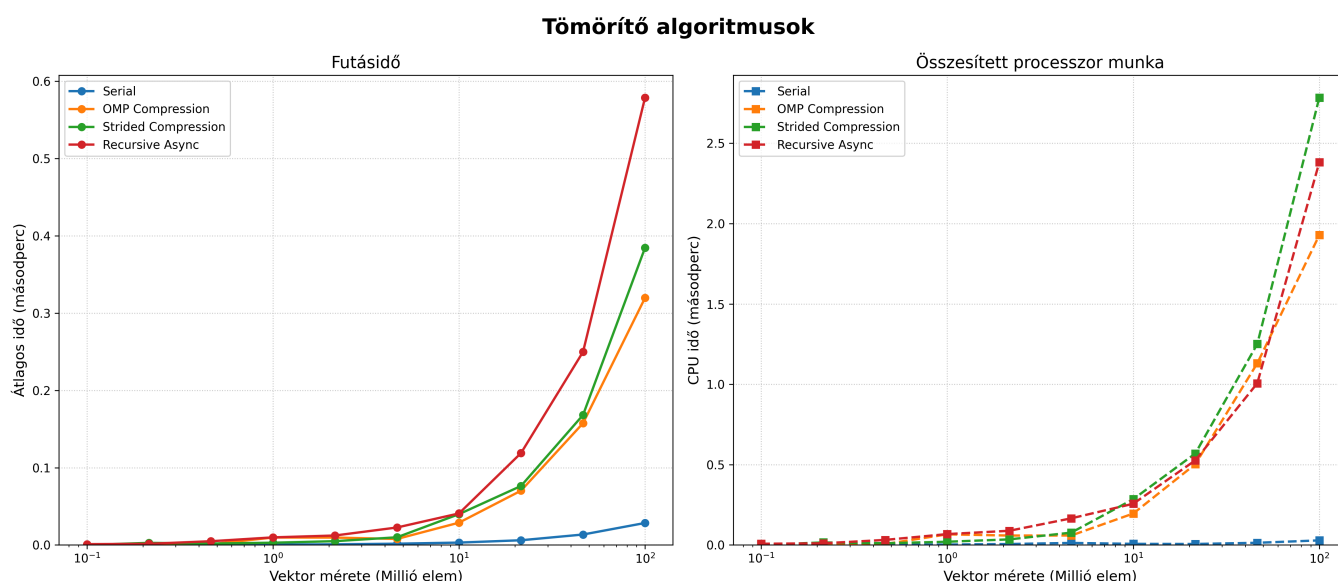
- Komplexitás:
 - Rekurziószám: $\log_2(n)$
 - Mértani sorozat: $n/2 + n/4 + \dots + 1 \approx n - 1$. Összes elvégzett munka: $O(n)$
 - Összesen: $O\left(\frac{n}{k} + \log n\right)$
- **Leírás:** Páronként összehasonlítjuk az elemeket, és csak a kisebbet tartjuk meg. Ezt a megfelelést iteratívan vagy rekurzívan addig folytatjuk, amíg egyetlen elem marad.
- **Buffer csere Tömörítés (OmpCompression)**
 - *Leírás:* Két vektort (**bufferA** és **bufferB**) használ, amelyek a rekurziós lépések során szerepet cserélnek: az egyikből csak olvasnak a szálak, a másikba csak írnak.
 - *Tervezői döntések:* * **Memória foglалás minimalizálása:** Csak a legelső lépésben történik Heap allokáció. A ciklusok közbeni folytonos **std::vector** létrehozás drasztikusan lelassítaná a programot.
 - **Szálbiztonság (Data Race prevenció):** A szerepcsere elv garantálja, hogy a szálak szigorúan különálló memóriaterületekről olvasnak és írnak, elkerülve a Read-Write Hazardokat.
- **Lépésközö hosszabító In-place Tömörítés (StridedCompression)**
 - *Leírás:* Egyetlen tömböt (in-place) használ, a külső ciklus a lépésközt (stride) duplázza (1, 2, 4, 8...). A belső OpenMP ciklus a **values[i]** és **values[i + stride]** értékeket hasonlítja össze.
 - *Tervezői döntések:* * Memóriaigénye tökéletes ($O(1)$).
 - **Hardveres anomáliák bemutatása:** Bár az elméleti aszimptotikus komplexitása kiváló, a gyakorlatban a kis lépésközöknél a **False Sharing** (hamis megosztás a L1 cache vonalakon), nagy lépésközöknél pedig a **Cache Misses** (memória ugrálások) drasztikusan lerontják a hatékonyságát.
- **Rekurzív Async (AsyncCompression)**
 - *Leírás:* Oszd-meg-és-uralkodj (Divide and Conquer) elv. A tömböt virtuálisan megfelezi, az egyik felét egy **std::async** aszinkron feladatként elindítja, a másik felét maga folytatja, majd összehasonlítja a két részeredményt.
 - *Tervezői döntések:* A **Küszöbérték (Threshold)** jelenléte kritikus. Ha az algoritmus az 1-es tömbméretig menne le aszinkron hívásokkal, a több százezer operációs-rendszer szintű szál létrehozása azonnal *Thread Bomb*-ot (erőforrás kimerülést / Out of Memory-t) okozna.
- **ThreadPool Pairwise (ThreadPoolPairwise)**

- *Leírás:* Egy `std::deque`-ből kivett elempárokat csomagol be `std::promise`-okba és küldi be a feladatsorba (enqueue), hogy a ThreadPool feldolgozza őket.
- *Tervezői döntések (Anti-pattern esettanulmány):* Annak bemutatására szolgál, hogy a túl finom szemcsézettségű (fine-grained) párhuzamosítás hogyan teszi tönkre a futást. Millió elemszámnál a memóriában létrejövő rengeteg `std::future` objektum és a Task Queue lezárásainak adminisztrációs költsége nagyságrendekkel tovább tart, mint maga a matematikai művelet, és hatalmas méreteknél OS-szintű omlást okoz.

• Mérések tanulságok és következtetések:

- Minél kevesebb vektort használjunk, kerüljük hogy minden rekurziós szinten létre kelljen hozni egy újat
 - Használjunk két vektort amit cserélgetünk, hogy melyik tárolja a jó eredményeket és melyik a bemeneti adatokat
 - Használjunk egy vektort, minden rekurzióban növeljük a lépéshosszát a bejárásának - lehetséges probléma az adatok írás miatt a chaseben lévő blokkok kiesése azok frissítése miatt
- Legyen egy határ (threshold) ami alatt sorosan oldjuk meg a feladatot

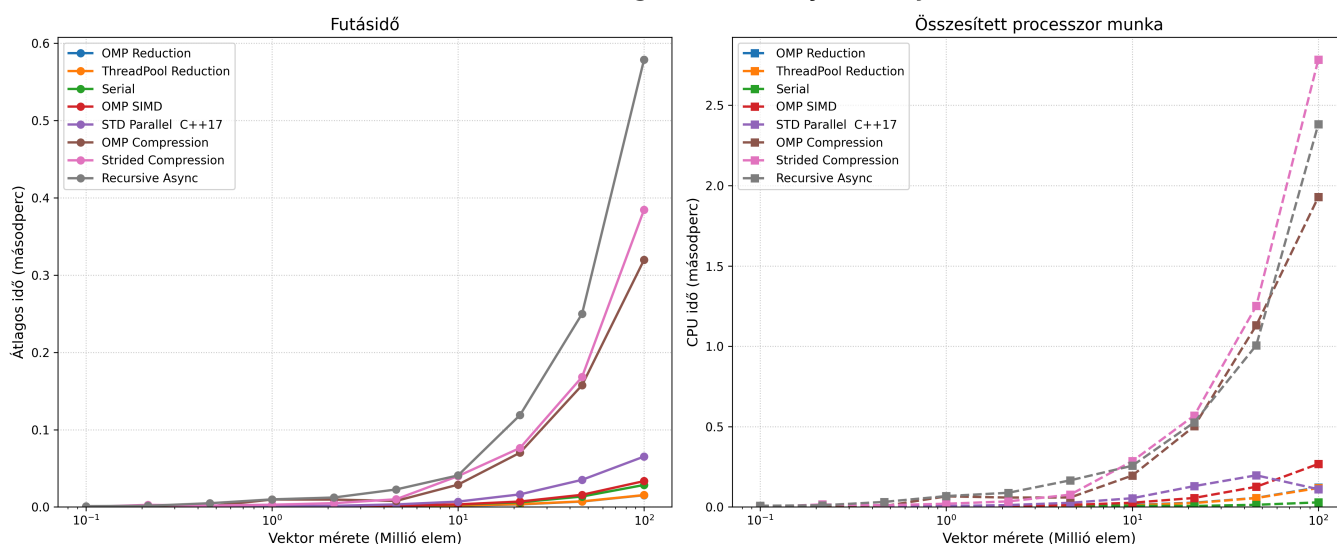
Mérések



- Jelentősen rosszabb hatékonyságúak mint a szeletelő algoritmusok - Egyik implementációnak sem sikerült elérnie a soros algoritmus hatékonyságát - A lépéshossz növelő megközelítés kicsit rosszabbul teljesített a szerepcserés buffer megközelítéshez képest (várhatóan a chase miatt) - A rekurzív aszinkron a sok taszk miatt ??? - A páronkénti hasonlítást nem jelenítettem meg a grafikonon mivel minden algoritmusnál jelentősen rosszabbul teljesített. Ahogy várható volt a túl kicsi feladatok párhuzamosítása (fine-grained) csak növeli az párhuzamosítási költséget (overheadet)

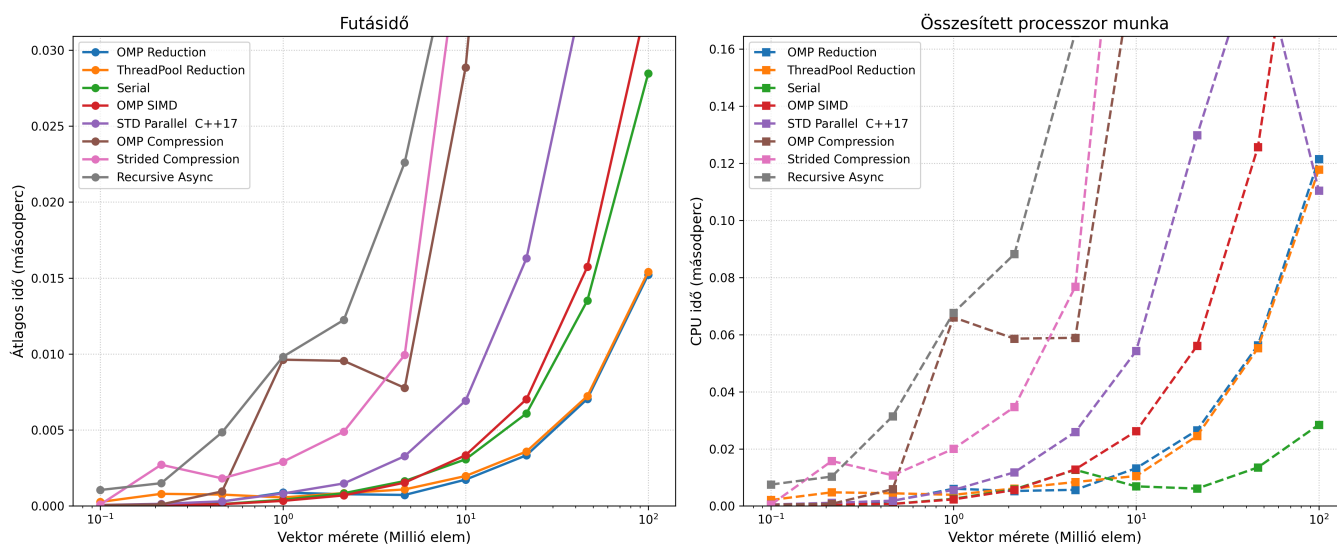
Konklúzió

Minimumkereső Algoritmusok Teljesítménye



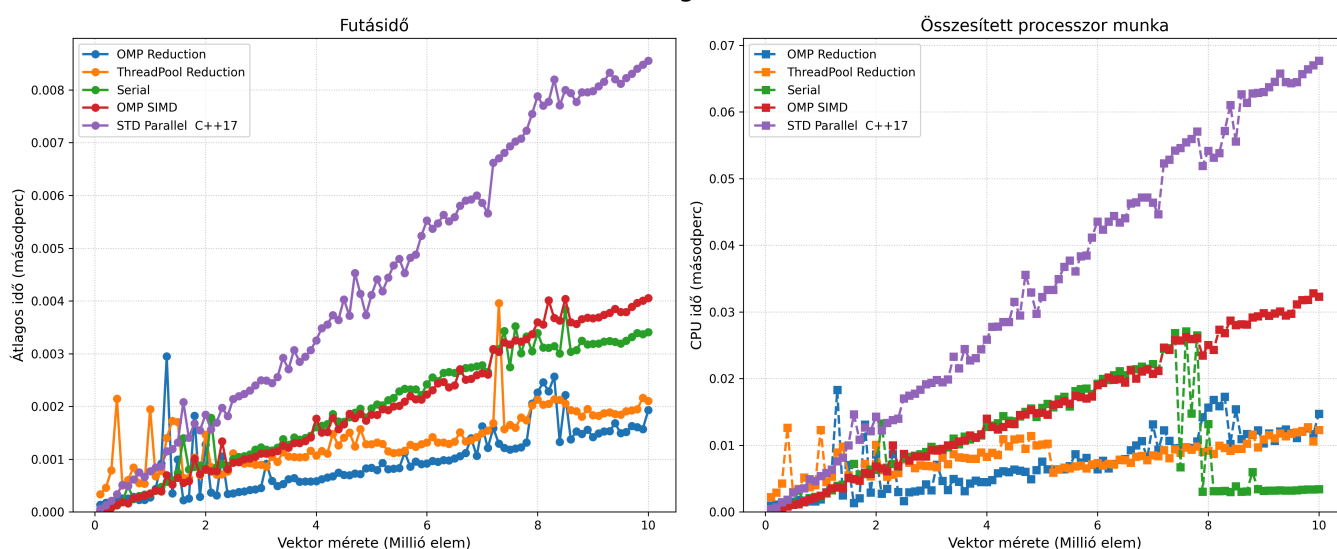
Az ábrán jól különválasztható a két algoritmikus megközelítés, de az implementáció is befolyásolja a hatékonyságát bár kisebb mértékben.

Minimumkereső Algoritmusok Teljesítménye



Jelentős gyorsítás nem vehetünk észre. Kb 2 szerez gyorsulás mutatkozik 100 millió elemű tömb esetén, ami a grafikunt figyelve nagyjából konstans 10 millió fölött, de lehetséges, hogy nagyobb méretek esetében még valamennyivel jobb relatív hatékonyságot el lehet érni.

Szeletelő algoritmusok



Nagyjából 200 és 400 ezer között veszik át gyorsaságban a vezetést a szeletelő algoritmusok. Érdekes még megfigyelni, hogy elég nagy kiugrások vannak a grafikonon még úgy is, hogy minden méretre 5-ször futattam az adott algoritmust és ennek az átlagát vettem fel a grafikonra.